Date

Concepts of Object Oriented Programming

1. Object (real world entity)
2. Class (user defined)
3. Polymorphism
4. Inheritance
5. Encapsulation
6. Dynamic Binding

Class - (Collection of Object)

A group of objects that share common properties for data part and some program part are collectively called as Class.

Collection of objects is called class.

We can create class by using class keyword.

There are three different mode of class -

1. Public
2. Private
3. Protected

The class binds together data and methods which work on that data.

```
Class student
{
    data ;  ⟶ Data member
    Methods ;  Member function
```

3. Polymorphism → Polymorphism feature enable classes to provide different implement of the same name.

Many → different forms

methods having → function

There are two types of Polymorphism,

① compile time → function overloading

```
float area (int b, int h)
{
    float a;
    a = b * h
    return a;
}
```

② Run time → operator overloading

```
float area (int r)
{
    float a;
    a = 22/7 * r * r;
    return a
}
```

Polymorphism

Inheritance :→ Inheritance is the process by which objects of one class acquire the property of another class.

It suppose the concept of hierarchical classification. It is a parent child concept in which one class is base class and another class is derived class as child class.

There are 5 different types of inheritance

① Single level inheritance
② Multi level inheritance.
③ multiple inheritance
④ Hierarchical inheritance
⑤ Hybrid inheritance.

Q:- Explain diff accessibility modi of class.

→ Three → public, private, protected.

simple class without constructor

```
class rectangle
{
private:
int l, b;

public:
int area ( );
{
return l*b;
}
};

int main()
{
int a;
rectangle r;

r.l = 12;
r.b = 16;
a = r.area ();
```

Constructor → Constructor is used to initialize the object of any class or the data members of any class.

Constructor is automatically called when we create object of any class.

The name of constructor is same as the name of class.

Class :- * Definition of class

```
class rectangle
{
private :          → Data member
int l, b;

public :  ← (const)  → const
rectangle ( int length, int breadth )
{
l = length ;
b = breadth;
};
  int area()   → member function.
  {
    return l * b;
  }
};

int main ()
{
  int a;
rectangle r (5,10);
a= r.area () ;  call.

cout << a;

}
```

Inheritance :-

The capability of a class to derive properties and characteristics from another class is called inheritance.

Inheritance is one of the most important feature of object Oriented Puggramming.

Child / derived / Sub class → The class that inherits properties of another class is called Sub/child/ derived class.

Parent / Super / Base class :→ The class whose properties are inherited by Sub class is called Base / Super class.

```
class car : public vehicle
{

};
```

Note :→ A derived class doesnot inherit access to private data member.

Visibility mode of Inheritance :-

1. Public mode :→ If we derive a sub class from a public base class then the public member of the base class will become public in derived class and the protected member will become

protective inderived class.

Protected mode

2. Private class :- If we derived a sub class, protected base class, then both the public and protected member of the base class will become protected in derived class.

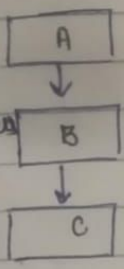
3. Private mode :→ If we ~~divide~~ derived a sub class from a private base class then both public members and protected member of base class will become private in derived class.

```
class A
{
public:
int a;
protected:
int b;
private:
int c;
};
```

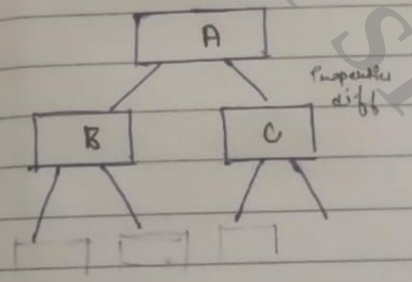
```
class B : privat A
{
a → private
b → private
c →
```

The private member in the base class cannot be directly excess in the derived class while protected member can directly be accessed.

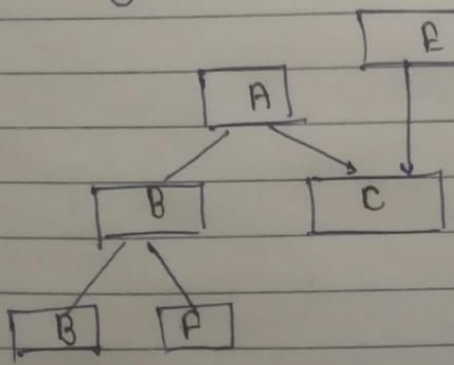
3. Multilevel inheritance :→ In this type of inheritance a sub class is ~~created~~ inherited from another derived/sub class



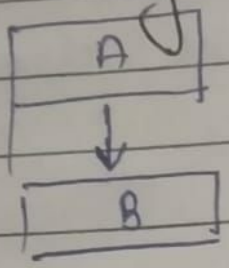
4. Hierarchical inheritance :→ In this type of inheritance more than one class is inherited from a single base class



5. Hybrid inheritance :→ Hybrid inheritance is implemented by combining more than one type of inheritances

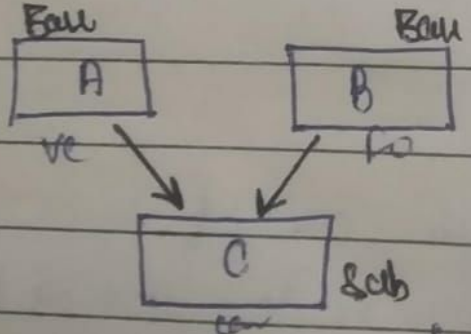


1. Single Inheritance :→ In single inheritance a class is allowed to inherit from only one class i.e One sub class is inherited by one Base class.

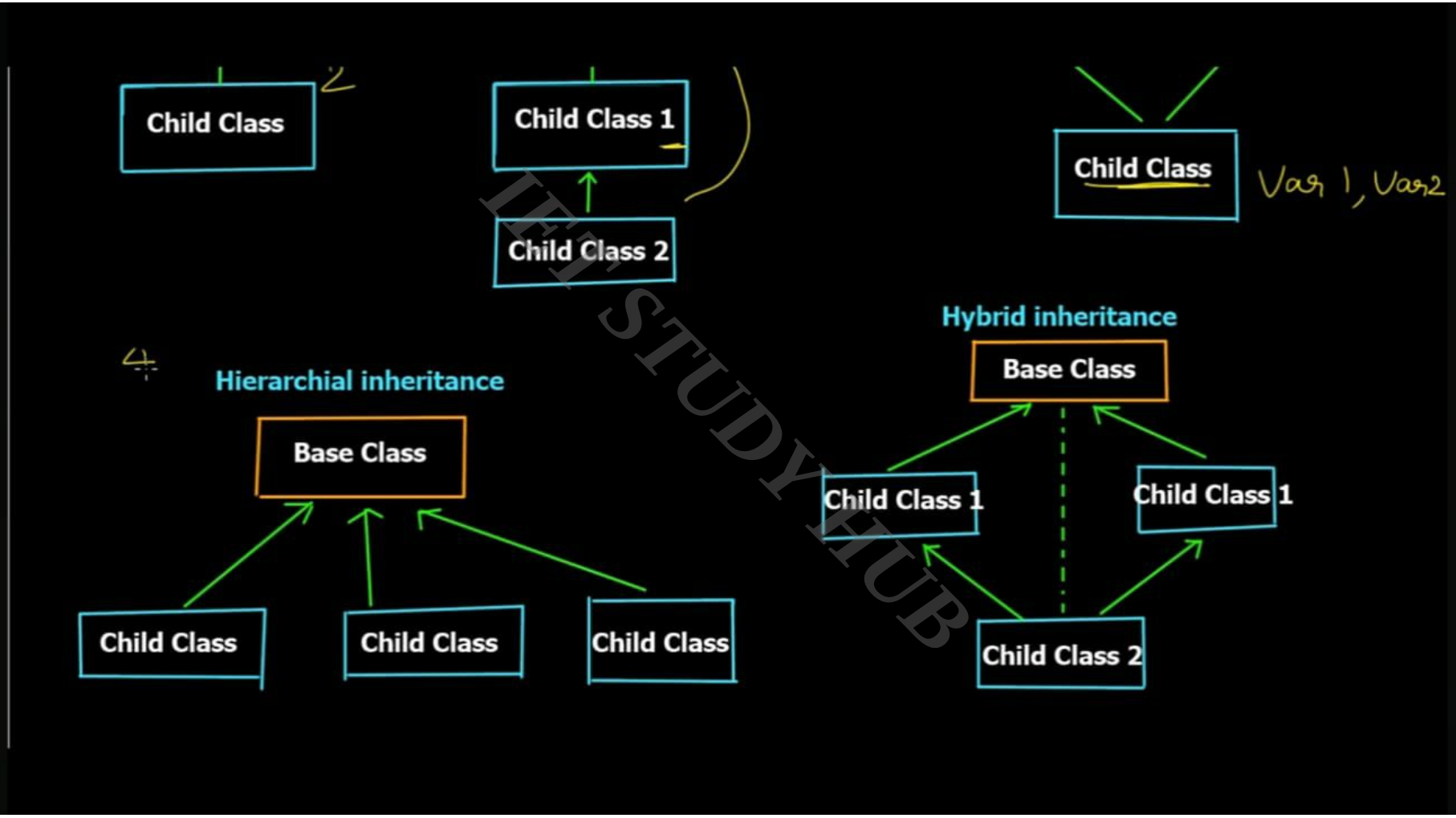


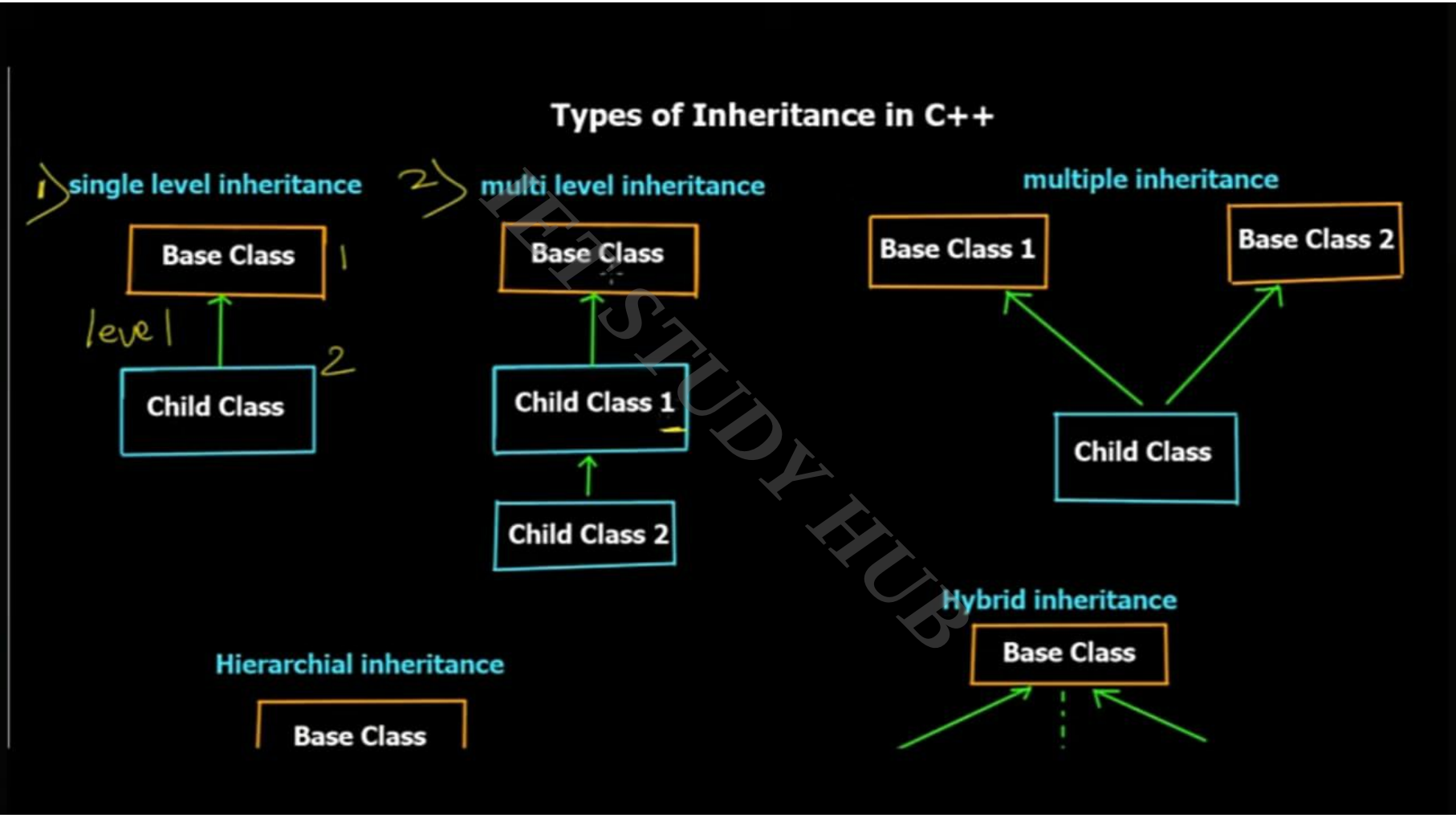
class subclass : access modes base class
{
//
//
};

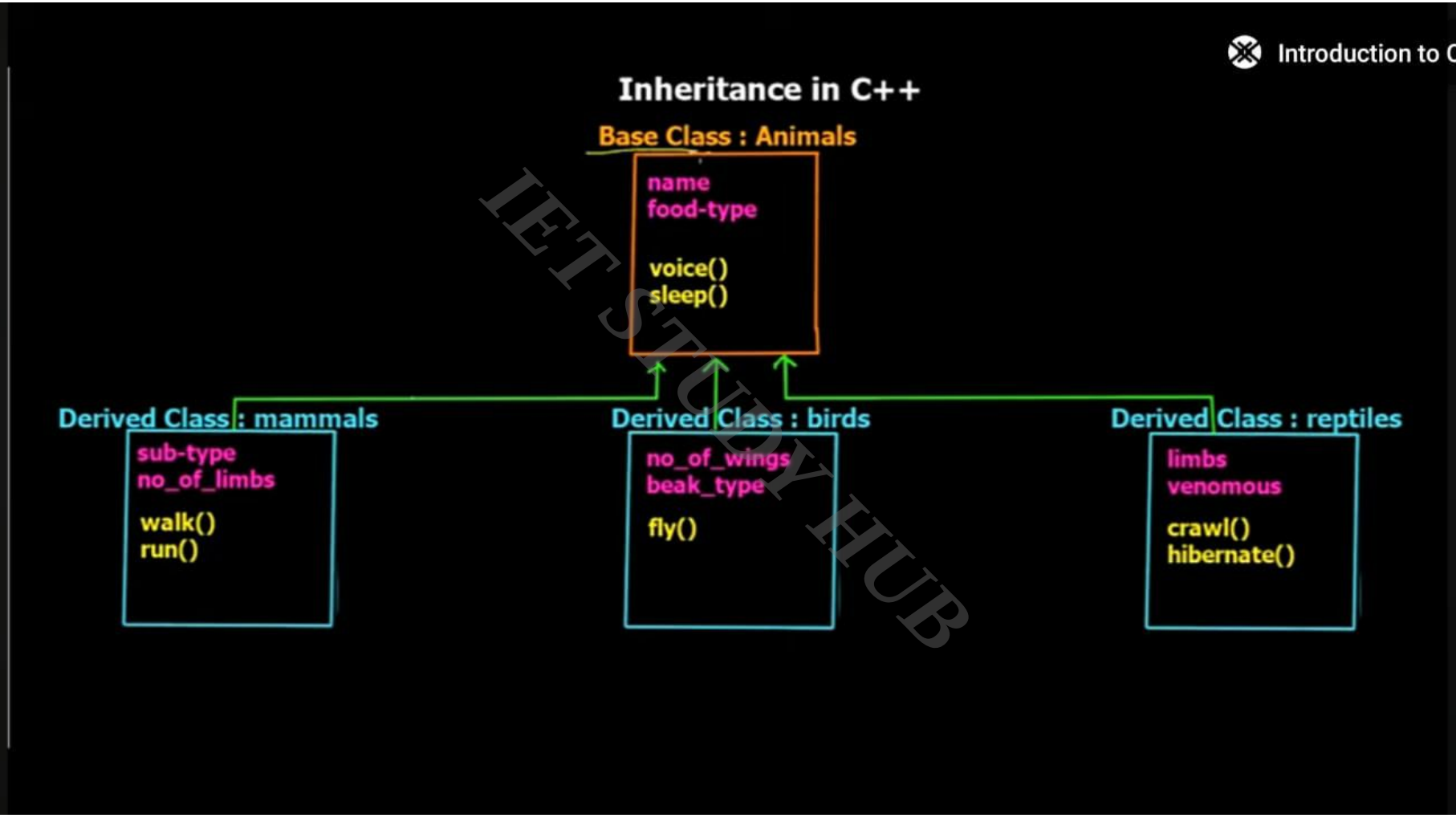
2. Multiple inheritance :→ In mutiple inheritance a class can inherit from more than one classes.



Class subclass : access modes base class, base class
{
};


Child Class
Child Class 1
Child Class 2
Child Class
Var 1, Var2
Hierarchial inheritance
Base Class
Child Class
Child Class
Child Class
Hybrid inheritance
Base Class
Child Class 1
Child Class 1
Child Class 2


Types of Inheritance in C++
1) single level inheritance
Base Class
1
level
2
Child Class
2) multi level inheritance
Base Class
Child Class 1
Child Class 2
multiple inheritance
Base Class 1
Base Class 2
Child Class
Hierarchial inheritance
Base Class
Hybrid inheritance
Base Class
IIT STUDY HUB


Introduction to Compu
Inheritance in C++
Base Class : Animals
name
food-type
voice()
sleep()
Derived Class : mammals
sub-type
no_of_limbs
walk()
run()
Derived Class : birds
no_of_wings
beak_type
fly()
Derived Class : reptiles
limbs
venomous
crawl()
hibernate()
IET STUDY HUB

Syntax of operator overloading

```
return_type Class_name :: operator Op(argument_list)
{
    // body of function
}
```

return-type is the type of value return by function.

Class-name is the name of the Class

operator Op is an operator function where Op is the operator being overloaded, and the operator is the keyword.

```
class base
{
public:
int x;
protected:
int y;
private:
int z;
};
class publicDerived: public base
{
// x is public
// y is protected
// z is not accessible from publicDerived
};
class protectedDerived: protected base
{
// x is protected
// y is protected
// z is not accessible from protectedDerived
};
class privateDerived: private base
{
// x is private
// y is private
// z is not accessible from privateDerived
}
```

Accessibility in Inheritance

Accessibility	private variables	protected variables	public variables
Accessible from own class?	yes	yes	yes
Accessible from derived class?	no	yes	yes
Accessible from 2nd derived class?	no	yes	yes

Accessibility in Public Inheritance

Accessible from own class?	yes	yes	yes
Accessible from own class?	no	yes	yes
Accessible from 2nd derived class?	no	yes	yes

Accessibility in Protected Inheritanc

Accessible from own class?	yes	yes	yes
Accessible from derived class?	no	yes	yes
Accessible from 2nd derived class?	no	no	no

Accessibility in Private Inheritance

Operator Overloading in C++

- C++ allows you to specify more than one definition for an operator in the same scope, which is called **operator overloading**.
- You can redefine or overload most of the built-in operators available in C++
- It is a type of polymorphism in which an operator is overloaded to give user defined meaning to it.
- Almost any operator can be overloaded in C++. However there are few operator which can not be overloaded. **Operator that are not overloaded** are follows-
  - scope operator (::)
  - sizeof
  - member selector –(.)
  - member pointer selector – (*)
  - ternary operator – (?:)

Syntax of operator overloading

```
return_type Class_name :: operator op(argument-list)
{
    // body of function
}
```

return-type is the type of value return b
function.

Class-name is the name of the Cl

or redefines most of the operators available in C++.

- Operator overloading is used to perform the operation on the use-define datatype.

- The advantages of operator overloading is to perform different operation on the same operand.

C++ Operator that Cannot be Overloaded.

⇒ Scope resolution operator (::)

⇒ Sizeof operator

⇒ member selection (.)

- Operator overloading is a Compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined datatype

- Operator overloading is used to overload or redefines most of the operators available in C++.
- Operator overloading is used to perform the operation on the use-define datatype.

The advantages of operator overloading is to perform different operation on the same operand.

C++ Operator that Cannot be Overloaded.

}

return-type is the type of value return by function.

Class-name is the name of the Class

operator op is an operator function where op is the operator being overloaded, and the operator is the keyword.

C++ Tutorials

Operator overloading

Operator overloading is a Compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined datatype

Operator overloading is used to overload or redefines most of the operators available in C++.
Operator overloading is used to perform the operation on the use-define data

The advantages of operator overloading is to perform different operation on